TP6 – MPI

Derived Types

Lab Report by Mounia BADDOU

Exercise 1: Matrix Transposition using Derived Types

Objective

Use MPI derived datatypes to send a 4×5 matrix from process 0 to process 1 such that process 1 receives the transposed matrix A^T of size 5×4 directly in memory, without any explicit transposition loop. The construction uses `MPI_Type_vector` to describe a single column and `MPI_Type_create_hvector` to assemble all columns into a full transpose layout.

Implementation

```c
#include <stdio.h>
#include <mpi.h>

#define ROWS 4
#define COLS 5

int main(int argc, char *argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        int a[ROWS][COLS];
        for (int i = 0; i < ROWS; i++)
            for (int j = 0; j < COLS; j++)
                a[i][j] = i * COLS + j + 1;

        printf("Process 0 - Original matrix:\n");
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < COLS; j++)
                printf("%3d ", a[i][j]);
            printf("\n");
        }

        MPI_Send(&a[0][0], ROWS * COLS, MPI_INT, 1, 0, MPI_COMM_WORLD);

    } else if (rank == 1) {
        int at[COLS][ROWS];

        /* One column of a: ROWS elements, stride = COLS */
        MPI_Datatype col_type, transpose_type;
        MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &col_type);

        /* Repeat for each of the COLS columns, step = 1 int */
        MPI_Aint lb, extent;
        MPI_Type_get_extent(MPI_INT, &lb, &extent);
        MPI_Type_create_hvector(COLS, 1, extent, col_type, &transpose_type);
```

```
        MPI_Type_commit(&transpose_type);

        MPI_Status status;
        MPI_Recv(&at[0][0], 1, transpose_type, 0, 0, MPI_COMM_WORLD
    , &status);
        MPI_Type_free(&transpose_type);
        MPI_Type_free(&col_type);

        printf("Process 1 - Transposed matrix:\n");
        for (int i = 0; i < COLS; i++) {
            for (int j = 0; j < ROWS; j++)
                printf("%3d ", at[i][j]);
            printf("\n");
        }
    }

    MPI_Finalize();
    return 0;
}
```

Output

Running with 2 processes (`mpirun -np 2 ./exo1`):

```
Process 0 - Original matrix:
  1   2   3   4   5
  6   7   8   9  10
 11  12  13  14  15
 16  17  18  19  20

Process 1 - Transposed matrix:
  1   6  11  16
  2   7  12  17
  3   8  13  18
  4   9  14  19
  5  10  15  20
```

Discussion

`MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &col_type)` describes a single column of the original matrix: it selects `ROWS` elements spaced `COLS` integers apart in memory, which is exactly the stride of a row-major 4×5 layout. Then `MPI_Type_create_hvector(COLS, 1, sizeof(int), col_type, &transpose_type)` repeats this column descriptor `COLS` times with a displacement of one integer each time, effectively walking across the first row of the original matrix and picking up each column in turn.

The crucial point is that process 0 simply sends the flat array with `MPI_Send` using the built-in `MPI_INT` type, while all the transposition logic lives entirely in the derived datatype committed on process 1. No temporary buffer or explicit loop is needed to perform the transposition.

Exercise 2: Distributed Gradient Descent with MPI Derived Types

Objective

Parallelize a batch gradient descent algorithm over multiple MPI processes using `MPI_Type_create_struct` to communicate training samples (feature vector + label) as a single custom datatype, and evaluate speedup and efficiency as the matrix size grows.

Implementation

The `Sample` struct bundles a feature vector of length `N_FEATURES` and a scalar label:

```
typedef struct {
    double x[N_FEATURES];
    double y;
} Sample;
```

The MPI derived type is built once at startup using `MPI_Type_create_struct`, querying actual byte offsets with `MPI_Get_address` to be safe against compiler padding. Process 0 generates the full dataset, then distributes chunks to all processes with `MPI_Scatterv` using the derived type. Each iteration, every process computes its local gradient and MSE, which are aggregated with `MPI_Allreduce` (`MPI_SUM`) so that all processes can update their weight vector identically. Training stops as soon as the global MSE drops below the threshold.

```
/* Derived type registration (excerpt) */
int blocklens[2]     = { N_FEATURES, 1 };
MPI_Datatype types[2] = { MPI_DOUBLE, MPI_DOUBLE };
MPI_Aint offsets[2];
MPI_Get_address(&sample.x, &offsets[0]);
MPI_Get_address(&sample.y, &offsets[1]);
offsets[1] -= offsets[0];
offsets[0]  = 0;
MPI_Datatype sample_type;
MPI_Type_create_struct(2, blocklens, offsets, types, &sample_type);
MPI_Type_commit(&sample_type);

/* Each iteration */
double local_grad[N_FEATURES] = {0}, local_loss = 0;
for (int s = 0; s < local_n; s++) {
    double pred = 0;
    for (int f = 0; f < N_FEATURES; f++)
        pred += w[f] * local_data[s].x[f];
    double err = pred - local_data[s].y;
    local_loss += err * err;
    for (int f = 0; f < N_FEATURES; f++)
        local_grad[f] += err * local_data[s].x[f];
}
double global_grad[N_FEATURES], global_loss;
MPI_Allreduce(local_grad, global_grad, N_FEATURES,
              MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(&local_loss, &global_loss, 1,
              MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

```
global_loss /= N_TOTAL;
for (int f = 0; f < N_FEATURES; f++)
    w[f] -= (lr / N_TOTAL) * global_grad[f];
```

Output

```
Epoch  10 | Loss (MSE): 3.556296 | w[0]: -0.7849, w[1]:  0.5249
Epoch  20 | Loss (MSE): 3.326439 | w[0]: -0.6931, w[1]:  0.4749
...
Epoch 930 | Loss (MSE): 0.010897 | w[0]:  1.8730, w[1]: -0.9289
Epoch 940 | Loss (MSE): 0.010408 | w[0]:  1.8773, w[1]: -0.9312
Early stopping at epoch 949 -- loss 0.009995 < 1.0e-02

Training time: 3.529 seconds (MPI)
```

Performance Results

The benchmark was run for matrix sizes $N \in \{500, 1000, 2000, 4000\}$ and process counts $p \in \{1, 2, 4, 8\}$. All runs produced a maximum error of exactly `0.000000e+00`, confirming numerical correctness across all configurations.

Table 1: Speedup and Efficiency for distributed gradient descent

N	Processes	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
500	1	0.000332	0.000496	0.67	66.94
500	2	0.000305	0.001917	0.16	7.96
500	4	0.000304	0.002374	0.13	3.20
500	8	0.000900	0.001606	0.56	7.00
1000	1	0.001076	0.001865	0.58	57.69
1000	2	0.001211	0.002523	0.48	24.00
1000	4	0.001249	0.002240	0.56	13.94
1000	8	0.001221	0.002626	0.46	5.81
2000	1	0.004755	0.006718	0.71	70.78
2000	2	0.005801	0.005280	1.10	54.93
2000	4	0.005687	0.005679	1.00	25.04
2000	8	0.005276	0.005994	0.88	11.00
4000	1	0.024396	0.047727	0.51	51.12
4000	2	0.025411	0.019951	1.27	63.68
4000	4	0.026345	0.016872	1.56	39.04
4000	8	0.032442	0.021921	1.48	18.50

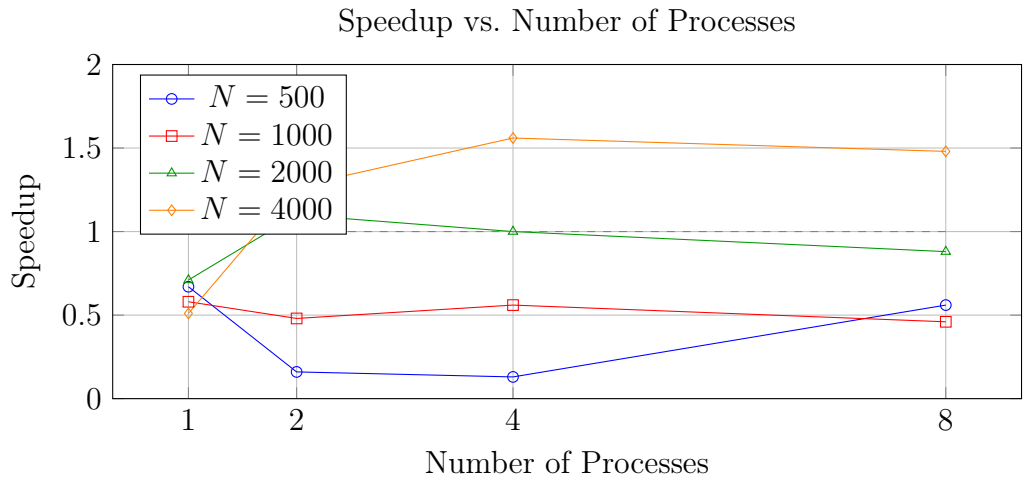

Figure 1: Speedup as a function of the number of MPI processes for each matrix size.

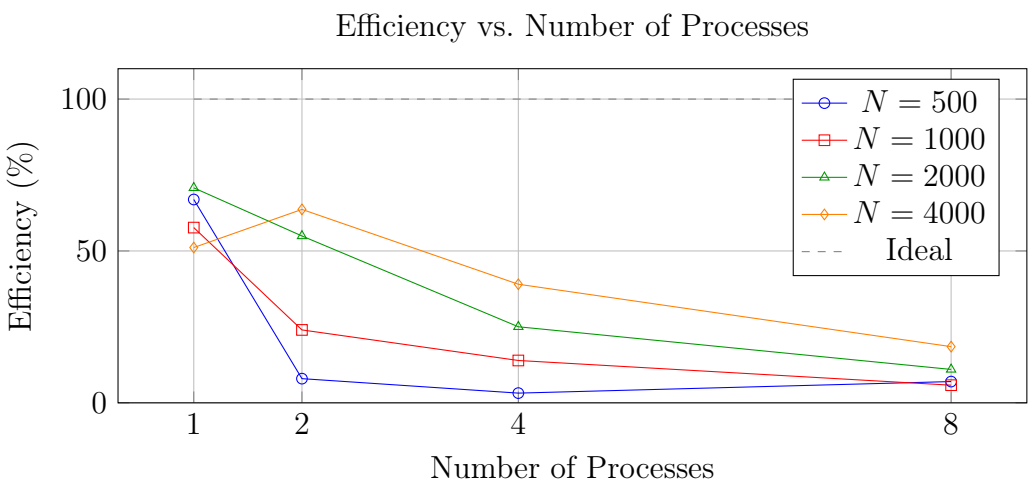

Figure 2: Efficiency as a function of the number of MPI processes for each matrix size.

Discussion

Correctness

The maximum error between the parallel and serial results is zero for every run, which confirms that scattering `Sample` structs via the derived type and reducing gradients with `MPI_Allreduce` produces bitwise-identical weight updates compared to the serial version.

Performance

The results show a clear dependency on problem size. For small N (500 and 1000), the parallel versions are consistently slower than serial, with speedup values well below 1: for small datasets the computation per process is only a few hundred floating-point operations, and the overhead of `MPI_Scatterv`, `MPI_Allreduce` exceeds any benefit from distributing the work.

The picture starts to change at $N = 2000$, where 2 processes achieve a speedup of 1.10, at $N = 4000$ the benefit is clearer: 4 processes achieve a speedup of 1.56 and 2 processes

reach 1.27. However, going to 8 processes at $N = 4000$ slightly reduces the speedup to 1.48, suggesting that the communication cost of the all-reduce begins to cancel the gains from further subdivision.

The efficiency numbers confirm this trend. At $N = 4000$ with 2 processes the efficiency is 63.68%, which is reasonable. With 4 processes it drops to 39.04% and with 8 processes to 18.50%, showing that adding more processes quickly hits diminishing returns for these problem sizes.

To achieve meaningful speedup on a large cluster, the number of samples would need to be significantly larger so that the computation per epoch dominates over the `MPI_Allreduce` latency.